12 - Hash Tables

A hash table is an array of structures as illustrated in Figure 12.1. The structures referenced by the array are typically linked-lists, but more sophisticated structures like trees could be used instead. The type of container each bucket references is an implementation detail left up to the hash table designer, so for the duration of this discussion, we will assume that a hash table is an array of linked lists.

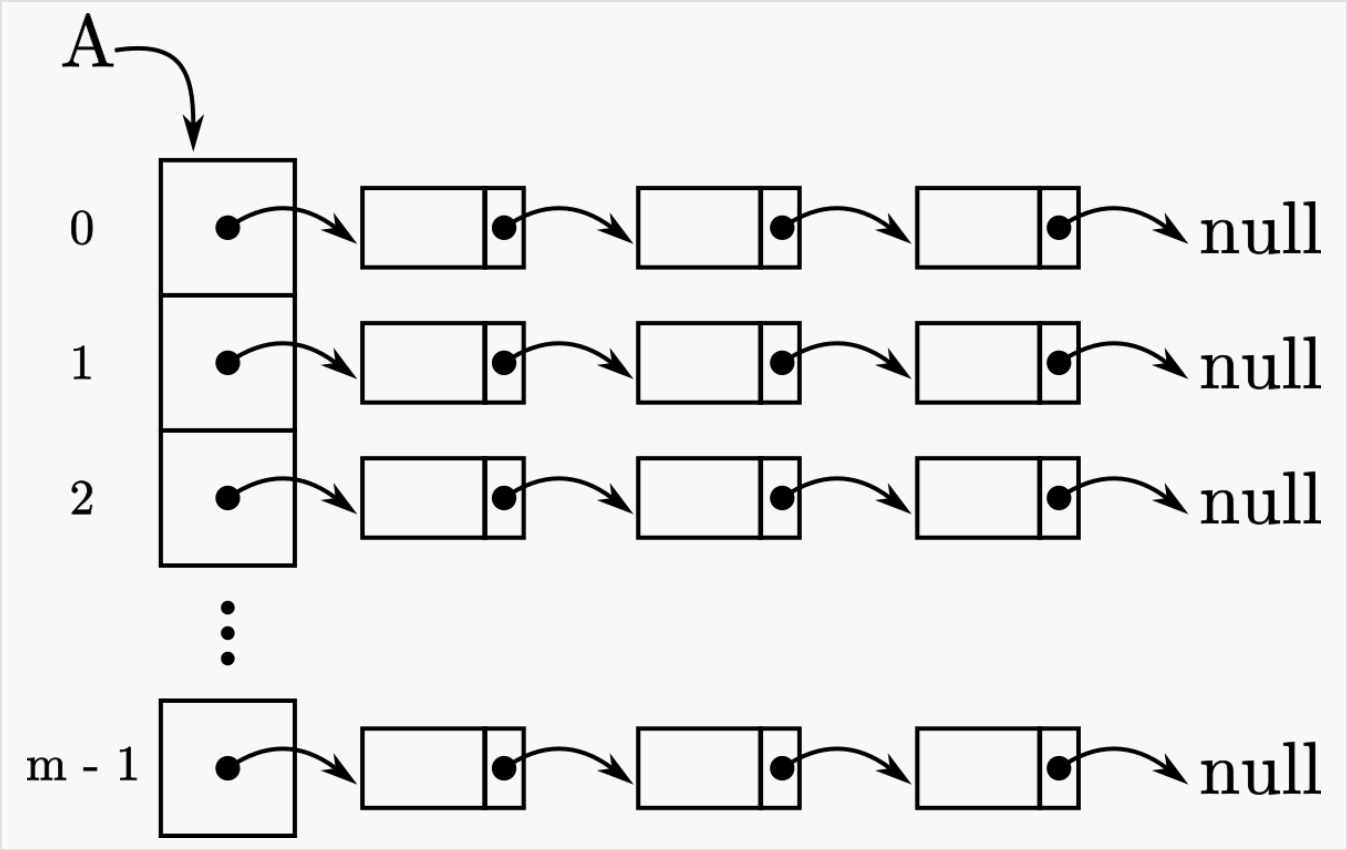


Figure 12.1 - Hash table A is an array of linked lists.

We call each index of the array a **bucket** to convey the idea that each index is itself a container that can hold multiple items. The number of buckets will be quantified by the variable m, so the size of the array is m; however, the size of the array is an independent decision from the size of the input n.

A developer controls the size of the array which is often static and determined at design time while users control the size of the input which is generally dynamic and determined at run-time. The linked-lists allow the hash table to dynamically grow to accommodate users but changing the number of linked lists (the size of the array) at runtime would be extremely inefficient, so the number of buckets is generally fixed at design time by the developer. It is rare to include logic that resizes the hash table programmatically during run-time because doing so effectively eliminates the possibility of having an insert efficiency better than linear.

Hash tables therefore introduce two independent variables that will affect efficiency, m and n, so we will need to consider and address both when performing any efficiency analysis on hash tables. However, we still want to conclude our overall efficiency of hash tables in terms of n exclusively, so that we can easily relate and compare the hash table to all other data structures and algorithms we have so far studied. We will need to classify hash tables according to a few general relationships between m and n so that we can effectively generalize our claims in terms of n in the final analysis.

To determine which linked list in the array a particular element should be associated with, we use a hash function.

Hash Function

The hash function's role is to map an **arbitrary size value** into a **fixed size value**. A hash function takes in a string of data as input and converts that input string into a single number that is bounded within a limited range of values. In the case of a hash table, the bounds are $[0, m-1]$ where m represents the number of buckets or the size of the array. If the range of outputs from the hash function is bounded $[0, m-1]$, then any output from the hash function would be a legal index for an array of size n. If the hash function is limited to this output range, then the hash function should convert any arbitrary size string into a legal fixed size index for the array.

In other words, given an input string the hash function converts this input string into an index identifying a bucket (the array element). which acts as the identifier of the specific linked list in the array that should contain any data associated with that input string. A given string always maps to the same bucket, so there is one and only one bucket that a given key may be associated with. Ideally, all buckets have sparse numbers of elements in them so looking in any bucket is a simple, constant time operation.

All values in computing are numeric, so all symbols in a computer are already mapped to numeric values anyway, *i.e.* ASCII, so performing some form of math on a string is trivial. Fixing an integer into a specific range is trivial using modulo, *i.e.* the remainder from integer division. The number of buckets, m, is determined when the a rray is allocated, so m is a fixed and known value at all times for the array. It is also not necessary to be able to retrieve the original input from the hashed output -- we don't have to be able to retrieve the key for a given value -- so the hash function does not have to be reversible, but it must be **deterministic** meaning that there is no randomness in the hash function and the same input key must always map to the same output value.

So in terms of a hash table acting as a map (a hashmap), we can hash any key and find out which array index the key is associated with. The key always hashes to the same index as long as the size of the array stays the same, so the hash function consistently and always maps *any arbitrary key* to *one specific bucket* as long as the size of the array stays the same.

```
int hash(HashTable H, String s)
    for each ch in s:
        sum += ascii(ch)
    hashvalue = sum % H.buckets
    return hashvalue
```

A hash function is a very basic form of cryptography and researchers have developed a variety of hash functions; however, hash functions can be quite simple as long as they consistently generate the same output given a specific input.

Requirements

An ideal hash function must be as fast as possible -- run in constant time -- since it will be used in every hash table operation to identify a key's associated bucket. Being the case, this means that its efficiency is a factor in all other operations, so the efficiency of hashing could dominate all operations. If hashing is constant time, then it will not be the dominant term in the efficiency equations for map operations and is effectively ignored.

An ideal hash function is also **unbiased**. Bias means that a system has a tendency to prefer one thing over another. An unbiased hash function should produce a **uniform distribution** of outputs given any set of inputs where a uniform distribution means that every result has equal probability like a “fair coin” or a “fair die”. A hash table's efficiency is dependent upon statistical probability where we would like the distribution of elements among buckets to be the average $\frac{n}{m}$, but this behavior only arises if the hash function is fair and there is a relatively equal probability for each bucket to be the result of a given hashing input.

Probability

For a fair coin, the probability of heads to be 0.5 and the probability of tails to be 0.5.

Ⓗ Heads Ⓣ Tails

These probabilities are derived from the fact that there are two options and both are equally likely to occur, so the individual probabilities are all $\frac{1}{2}$. For a fair six-sided die, the probability of any one specific number of the die being rolled would be $\frac{1}{6}$.

Probability says nothing about the outcome of a single coin flip or a single die roll and probability is incapable of predicting the outcome of the next trial. Instead it is a measure of the aggregate total of many outcomes with respect to a significant number of trials. In other words, it is perfectly within the bounds of probability to flip a coin repeatedly and get a sequence of heads or a sequence of tails; however, if the coin is flipped a sufficient number of times, *i.e.* hundreds of times, all of the sequences will cancel each other out eventually and the ratio of the number of heads outcomes to the total number of trials will converge towards the predicted probability of $\frac{1}{2}$.

Unfortunately, what represents a "sufficient number" is ill defined and the answer is generally a lot more than you think you need. In other words, statistics is more meaningful, accurate, and predictive over large sample sizes so the more flips you make the closer the answer will get to the true probability.

Suppose we flipped a fair coin ten times and the result was a streak of heads one after another.

Ⓗ 10 Ⓣ 0

Ⓗ	Ⓗ	Ⓗ	Ⓗ	Ⓗ	Ⓗ	Ⓗ	Ⓗ	Ⓗ	Ⓗ
1	2	3	4	5	6	7	8	9	10

This is an extremely rare event and seems to defy probability; however, this event is possible and is equally probable to flipping a coin ten times and getting the precise sequence of heads-tails-heads-tails-heads-tails-heads-tails-heads-tails.

Ⓗ 5 Ⓣ 5

Ⓗ	Ⓣ	Ⓗ	Ⓣ	Ⓗ	Ⓣ	Ⓗ	Ⓣ	Ⓗ	Ⓣ
1	2	3	4	5	6	7	8	9	10

We enter the realm of permutations when specific sequences are focused on, and as discussed earlier, this means that the number of sequences possible from flipping a coin multiple times is measured in terms of a factorial equation. It also means that the probability of getting a specific sequence is very, very small because this is the only version out of hundreds, thousands, millions, etc. of permutations with that exact sequence.

The probability of a single flip is $\frac{1}{2}$ because there are only two options (heads or tails), both are equally likely (fair coin), and one of those outcomes must happen (cannot result in a "null" outcome like balancing on the edge of the coin). However, the probability of a specific sequence of flips is the product of all the probabilities for each individual flip which as the length of the sequences increases makes it increasingly less likely to have two sequences of flips equal each other. Each flip is its own independent sample so it is unbiased and unaffected by any previous states. So while it is highly probable to get a heads followed by a heads it is much less probable to get heads, heads, heads, or any other longer specific sequence.

Over the span of all flips of a fair coin, regardless of the specific sequencing, the coin should trend towards an equal number of heads and tails. Sometimes we can get a string of heads and it is even possible, if unlikely, to get ten heads in a row; however, if we flip the coin enough, the overall count of heads and tails will be approximately equal.

Most sequences of ten flips will be roughly equal in terms of the number of heads and tails

Ⓗ 4 Ⓣ 6

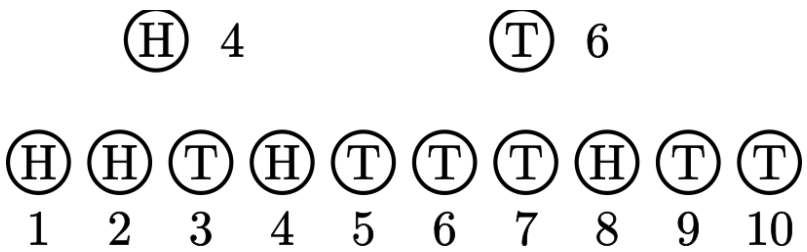
Ⓣ	Ⓗ	Ⓗ	Ⓣ	Ⓣ	Ⓗ	Ⓣ	Ⓗ	Ⓣ	Ⓣ
1	2	3	4	5	6	7	8	9	10

Some sequences may generate a large run of heads

Ⓗ 6 Ⓣ 4

Ⓣ	Ⓗ	Ⓗ	Ⓗ	Ⓗ	Ⓣ	Ⓣ	Ⓗ	Ⓣ	Ⓗ
1	2	3	4	5	6	7	8	9	10

But for every run of heads, there will be a run of tails or small runs of tails roughly canceling it out



In the end, for a fair coin with enough flips, we will find that there is approximately the same number of heads and tails. If a coin is flipped 1000 times, the outcome may be 506 heads to 494 tails or approximately a 50/50 ratio. If we continue to flip, the results will continue to trend towards the 50/50 ratio even if the numbers are not precisely 50/50. Probability is messy in terms of the specific details, but the overall statistics are consistent as long as the system is fair.

A good hash function should avoid the tendency to pick a subset of outputs over all other potential outputs and yields a probability of $\frac{1}{m}$ for any output index generated making the probability of any index generated both equal and fair given a randomly generated input key. An unbiased hash function means that It is okay for two or more keys to hash to the same bucket as long as the probability of hashing to any bucket remains equal. From a practical standpoint, this means that for a small number of inputs into a hash table (less than the number of buckets), there will be some buckets containing zero keys, most buckets containing one key, and a few buckets containing two or more keys. While some buckets may contain two or more keys, this should be extremely rare if the number of inputs is small. If one bucket contains a significant number of keys given a small number of inputs and a significant number of buckets, then it is probably indicating a biased hash function.

Implementation

A hash table is generally defined as an array that uses a hashing function to determine which index an element is assigned. A hash table is also an implementation of the map ADT, so a hash table fulfills the map interface and is often called a hashmap for this reason.

```
class HashMap {
    private LinkedList[] table;             // array of linkedlists
    private int hash( String key ) {...}    // hash function

    // public map interface
    put( key, value ) {...}
    value get( key ) {...}
    value delete( key ) {...}
}
```

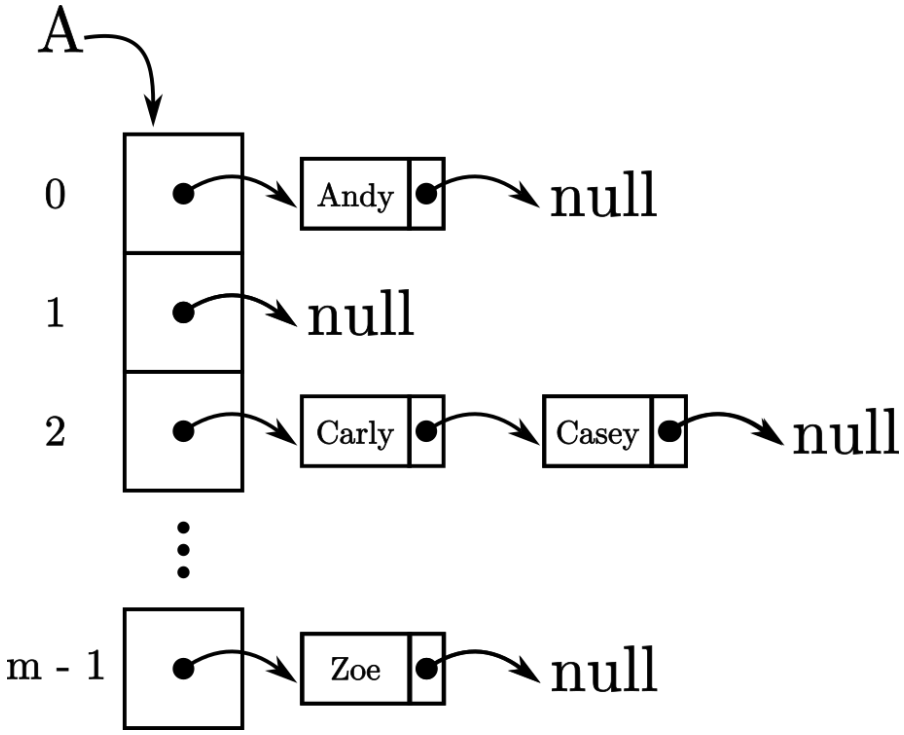
In each map function, the hash table first uses the hash function to hash the input key to find an integer representing the index of the bucket that the key-value pair must belong to. However, by definition of a map, the key must always remain unique within the hash table, so the bucket must be searched for the existence of the key in all operations.

For example, `put` performs either an insert or an update operation by definition. If the key exists in the bucket, then `put` performs an update, and if the key does not exist in the bucket, then `put` performs an insert. This means the existential question determines the operation performed by `put` so search must be performed first.

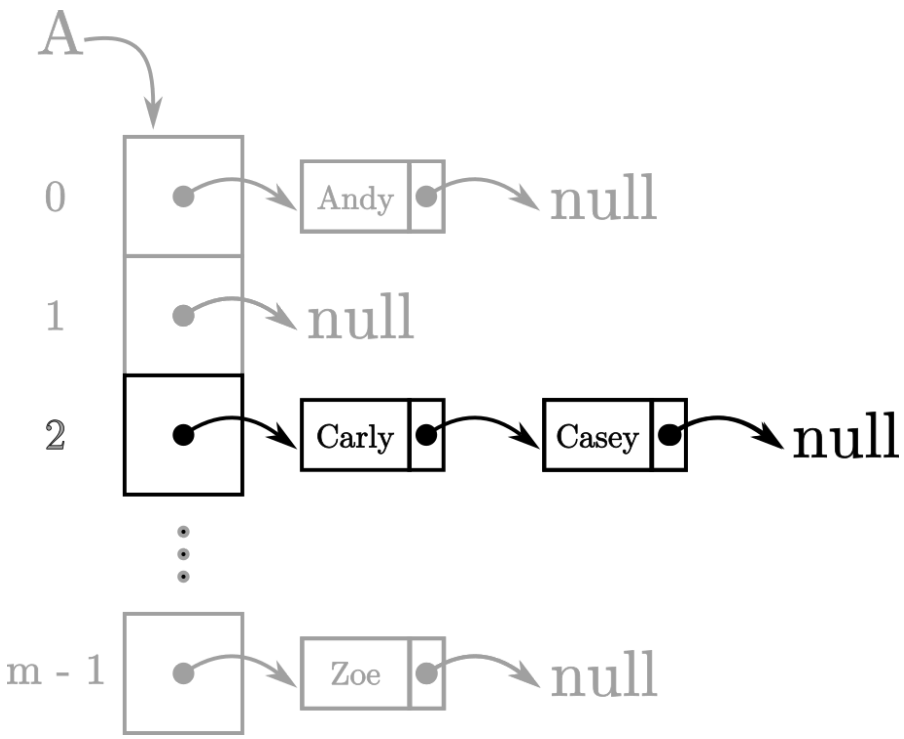
Hashing to a specific bucket is easy and fast, but searching a bucket will depend on how many elements are in the bucket. If the bucket is not empty, we cannot know whether or not we are inserting into the bucket until we traverse the entire linked list contained in that bucket. For this reason, it is mostly useless to have a tail associated with the hash table linked lists because we cannot blindly insert into a bucket and instead we must first traverse it and find the tail anyway.

Consider a concrete example of `put`. When the key-value [“Hello”, “World”] pair is put into the hash table, the key “Hello” is hashed, suppose the hash function returns the result of 2. The bucket at index 2 is searched for the key “Hello”. If the key is found, the existing value associated with the key-value pair in the bucket is updated to “World” and the operation is complete. If the key is not found which can only be known if the tail of the linked list in bucket 2 is reached, then a new key-value pair is appended to the end of the list in bucket 2.

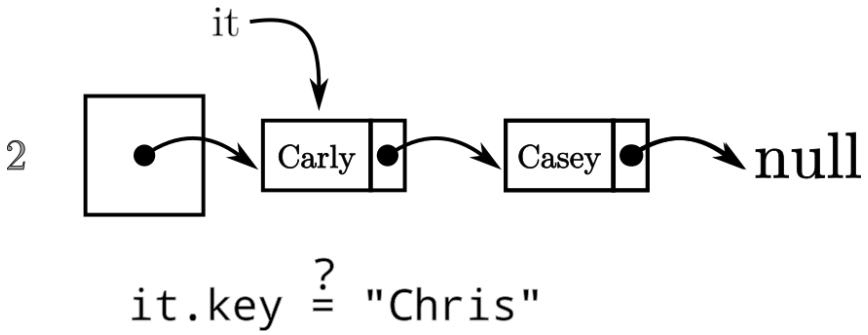
Consider a concrete example of `put` involving a hash table containing keys representing first names with 26 buckets and the hash function returning the index associated with the alphabet position of the first character in the first name. For example, the key "Andy" would hash to the first bucket because it begins with "A", the first letter in the alphabet and the key "Casey" would hash to the third bucket because it begins with "C", the third letter in the alphabet.



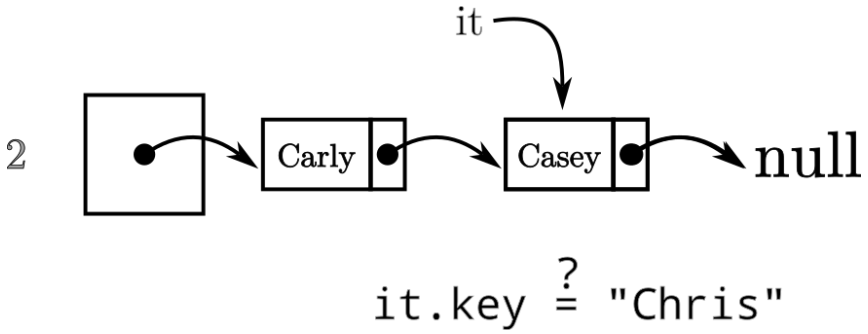
If we insert the key "Chris" and some associated value data into the above hash table, we would first hash the key to find "Chris" should belong to the linked list associated with index 2.



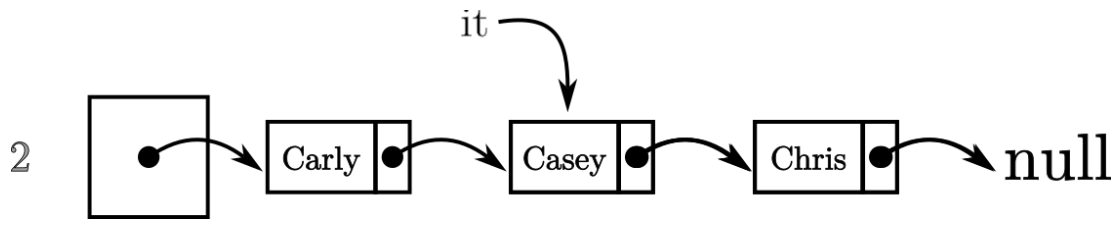
We would then need to iterate over the linked list referenced by index 2 comparing each key in the linked list to the to the search key starting with the first element.



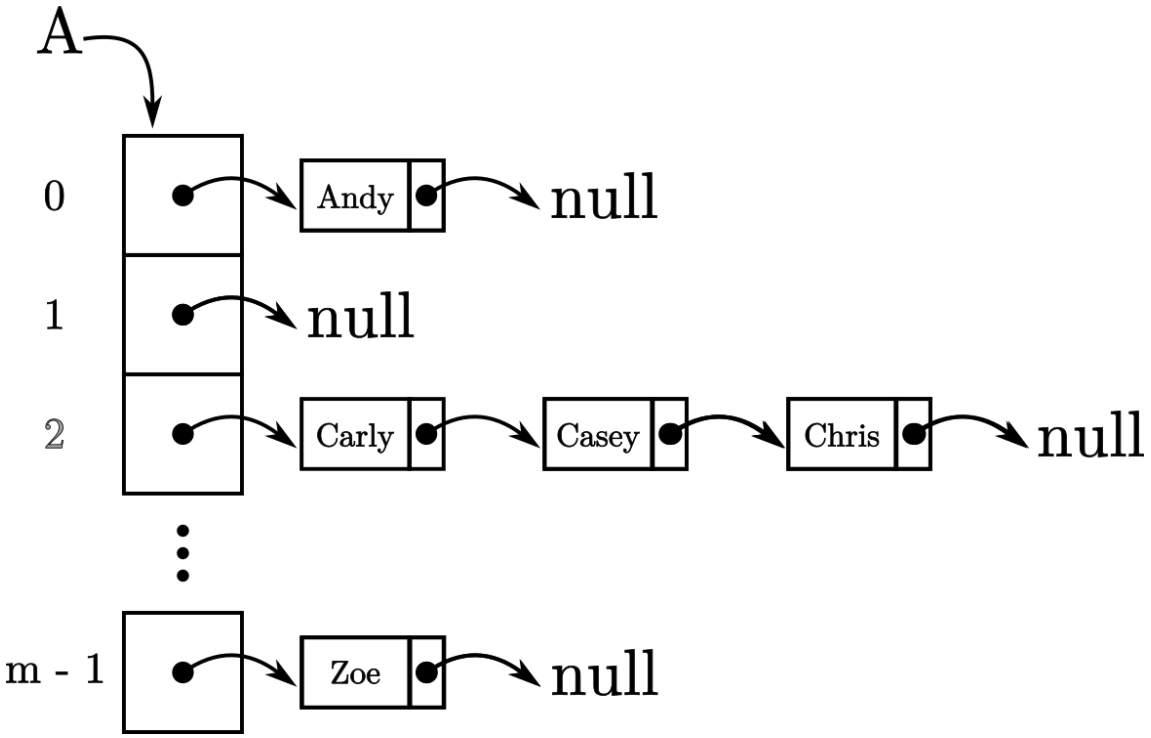
We would end the search if the key was matched OR if we reached the end of the linked list.



If the last element is compared and it does not contain the key that is being searched for, the key and associated data can be inserted as a node at the end of that linked list.



By adding the element to the end of the linked list designated by the hash function, the element is added to the overall hash table



Otherwise, if the key was found before the end of the list was reached, then `put` performs an update of the value at the element pointed to by the iterator.

Time Complexity

Remember that there are two independent variables that must be considered when analyzing the efficiency of the hash table: n, the number of elements **in the entire hash table**, and m, the number of buckets in the hash table.

While we must consider both variables at the beginning of the analysis, we want to eliminate m through our analysis so that we can continue to compare the hash table to all other data structures and algorithms. In other words, the value of m affects the efficiency a hash table so we cannot ignore m in our analysis, but we need to find a way to eliminate m in our conclusion so that we can establish a final Big O evaluation(s) involving only n. We want to frame the final analysis in terms of n because other algorithms and data structures do not have any correlation with m. If we our final Big O analysis includes both m and n, then hash tables cannot be easily and directly compared to other data structures. We did this in the tree analysis as well when we introduced the height of the tree as an analytical variable and eliminated it in the analysis by established and expressing the mathematical relationship between height and n in terms of n.1

Let us assume that the hash function is unbiased and consider the analysis of the hash table in terms of the ratio of number of elements in the hash table to the number of buckets, *i.e.* $\frac{n}{m}$. Structuring the ratio in this manner makes sense because a hash table always has at least one bucket, so the denominator is always a positive, non-zero number. Additionally, if we can somehow eliminate m through the analysis then n already resembles other Big O analysis.

The ratio of n and m is of such significance, it is specially named as the **load factor** and is a measure of "how full" a hash table is.

We can classify the hash tables as belonging to one of two cases:

1. n is significantly larger than m, *i.e.* $n >> m$
2. n is approximately equal to (or smaller than) m, *i.e.* $n \lesssim m$

Analysis of Case 1

Assuming an unbiased hash function, if n is significantly larger than m, *i.e.* $n >> m$, the numerator of the ratio $\frac{n}{m}$ dominates the equation. The size of each bucket on average is trending towards the overall number of elements in the entire table or the number of elements in each bucket is approximately n elements. In other words, m is constant and small so we can decompose the equation $\frac{n}{m}$ into the equation:

$$\frac{1}{m} \cdot n$$

With $\frac{1}{m}$ acting as a constant coefficient, we can drop constant coefficients in Big O analysis so the equation reduces to:

$$n$$

And as a result, every search is effectively like searching a linked list of size n and therefore the overall efficiency of the hash table is $O(n)$. Since we must search in every map operation, the efficiency of `put`, `get`, and `delete` are all $O(n)$.

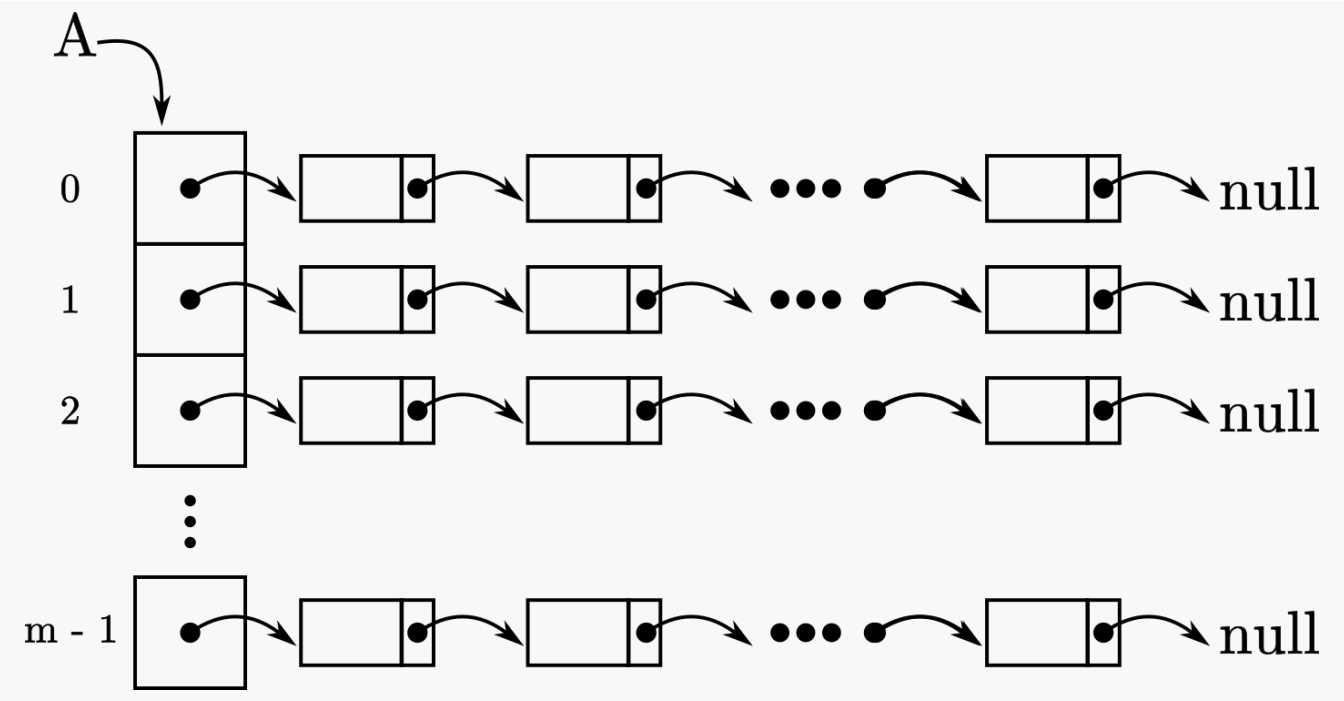


Figure 2 - A hash table that performs in linear time has significantly more data than buckets and therefore all the buckets are performing more like linked lists

This case represents the scenario where a user has crammed a lot of data into the hash table and we the developer did not anticipate the hash table would be supporting this much data when we specified the size of the hash table initially.

Analysis of Case 2

Assuming an unbiased hash function, if n is approximately equal to (or smaller than) m,

$$\lim_{n \to m} \frac{n}{m} = 1$$

In other words, as the ratio $\frac{n}{m}$ approaches 1, n and m are approximately equal and $\frac{n}{m} \approx 1$. Under these circumstances, it is reasonable to conclude that the efficiency of the hash table is $O(1)$ and constant time.

In fact, for any n where $n < m$, then $\frac{n}{m} < 1$, and $O(1)$ continues to be a reasonable, straightforward conclusion.

In this case assuming an unbiased hashing function, each bucket contains about 1 element. Some might contain two elements, some may contain zero, but from a statistical standpoint all buckets contain about the same number of elements and that number is small and approaches 1. In this case, buckets are mostly empty or contain only one or two elements each, and the efficiency of every search is effectively like searching an array using direct access because each bucket has an average size of 1 and the maximum number of elements per bucket approaches an average of 1. Therefore the overall efficiency of the hash table is $O(1)$ for every map operation.

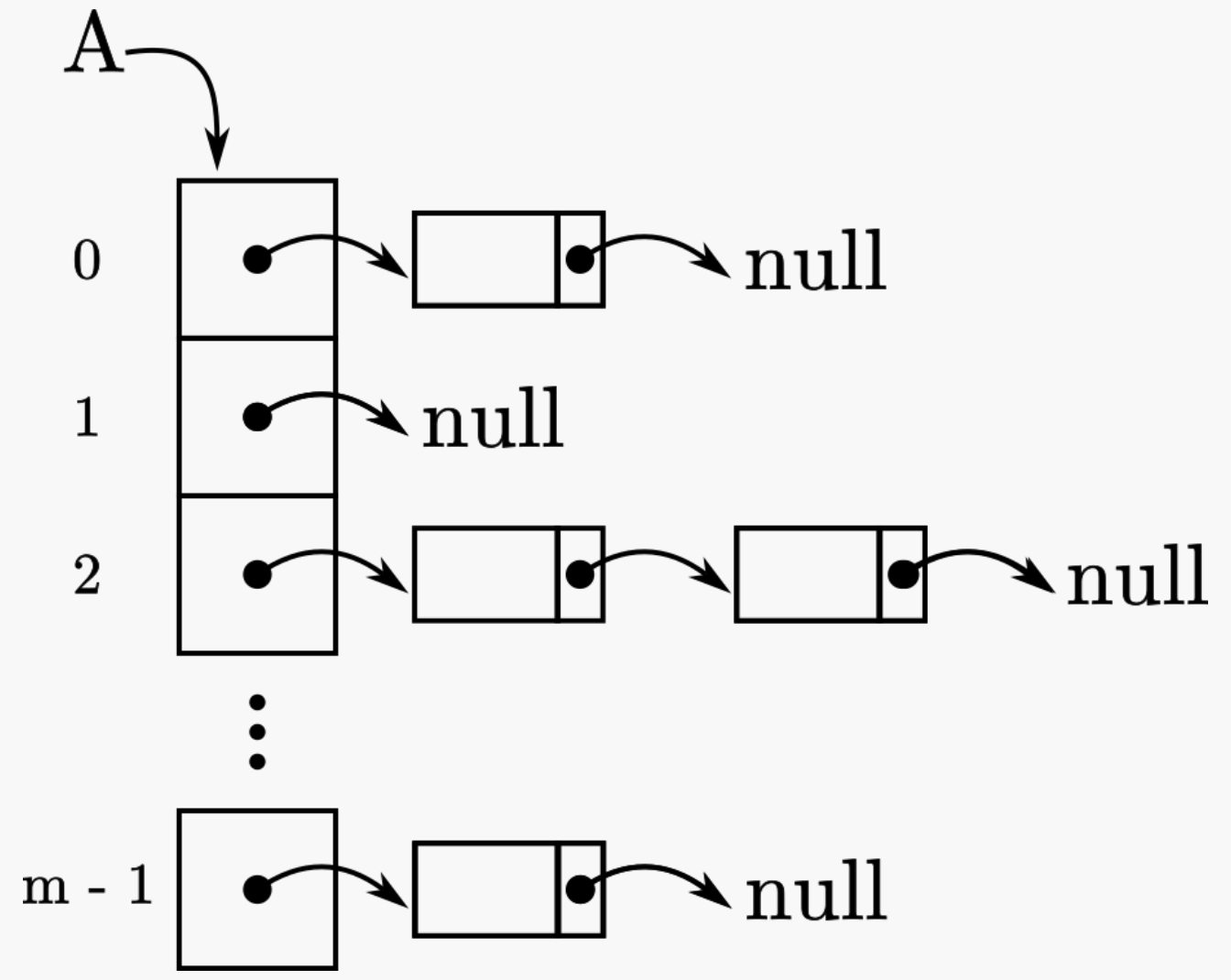


Figure 3 - A hash table that performs in constant time has a near uniform distribution of elements among m buckets and each bucket has a small number of elements in it.

This does raise the question of when does the efficiency tip from this case to the first case and that is a difficult answer and hence the reason the comparison for the first case is "n is significantly larger than m".

This second case represents the scenario where the developer has appropriately studied and modeled their data needs and has a reasonable understanding of the maximum amount of data that will need to be stored in the map. This does not mean that m must be larger than n to achieve constant time performance, but it does mean that the developer has a reasonable expectation of the upper bound on the number of elements that should be in each bucket.

Analysis of Degenerate Case(s)

Like a degenerate tree, we can have degenerate cases arise with a hash table. Degenerate cases are generally failures of the system and are not strictly due to insufficient design.

One such degenerate case that is out of the developers control can arise due to software being used for a long period of time without update. As software becomes more critical and trusted and more data become introduced into a problem space, users often start to exceed the original design specifications and performance can precipitously drop especially when the initial design assumed a maximum input size of n and users are currently trying to use the same software on inputs of size n^2 or worse.

Another degenerate case that is under the developer's control is the case of *bias in the hash function*. A biased hash function renders the statistical assumptions that we build our efficiency analysis on for a hash table moot. Our argument about hash table performance is built on the average number of elements per bucket being approximately $\frac{n}{m}$ which is linked to the requirement that the hash function must produce a uniform distribution of outputs. In other words, all outputs must have equal probability. However, if there is bias then one of the outputs is more likely than the others and the efficiency of the hash table is negatively affected.

If the hash function is biased, this can lead to one bucket being preferred over all others. This means that more keys hash to a single index and the linked list in that index grows significantly larger than all other indices. In this case, it is more likely that the hashing of any key will result in the access of the biased bucket and the number of elements in that bucket is trending towards n and therefore the required search for each map operation is now iterating over a linked list of size n rather a linked list of size nm. In other words, with a biased hash function, the hash table degenerates into performing like a linked list instead of a hash table and the efficiency of its operations are $O(n)$.

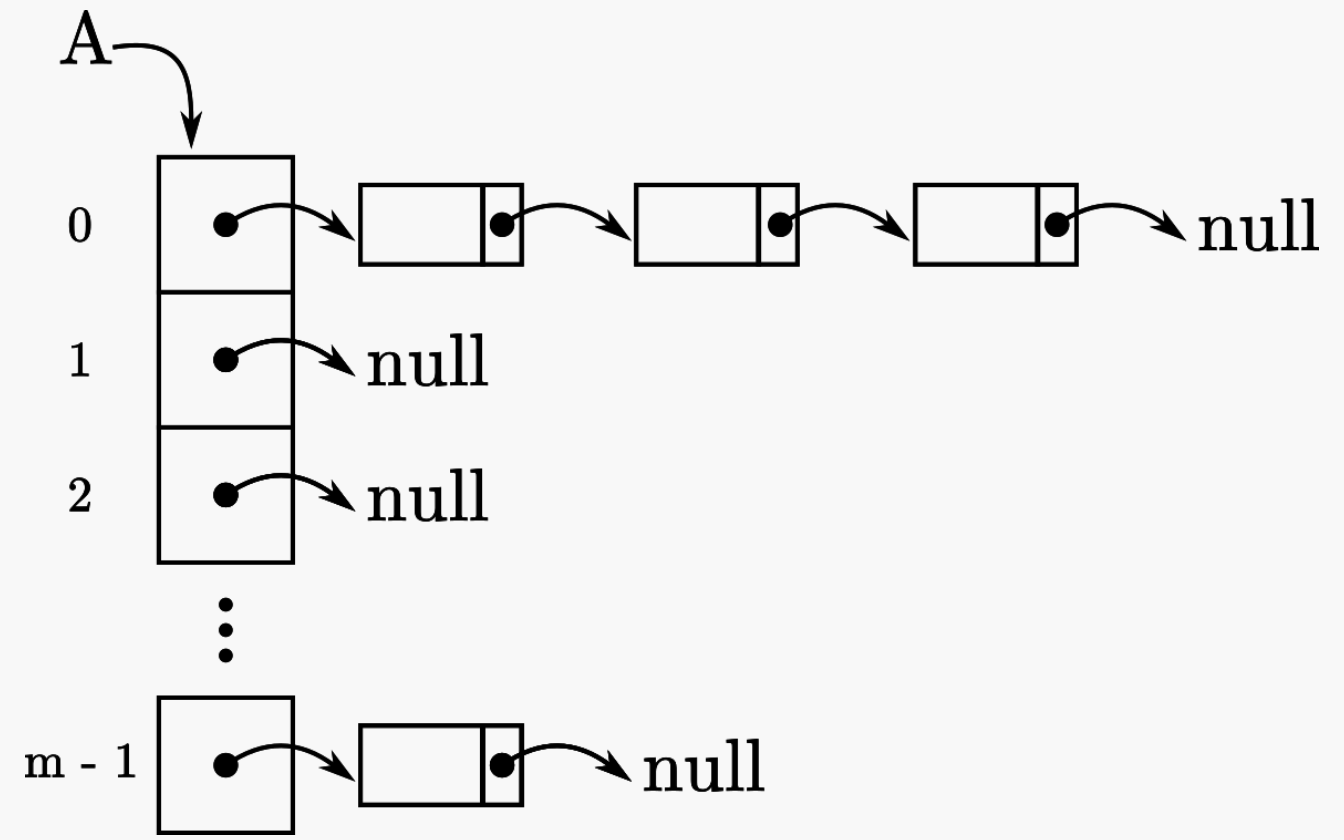
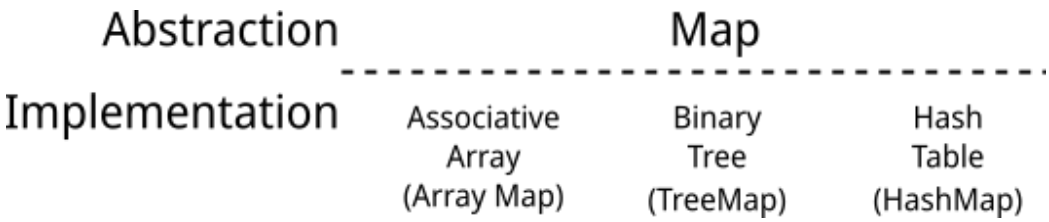


Figure 4 - A biased hash function results in a degenerate hash table where one bucket is preferred over all others.

Alternative Decisions

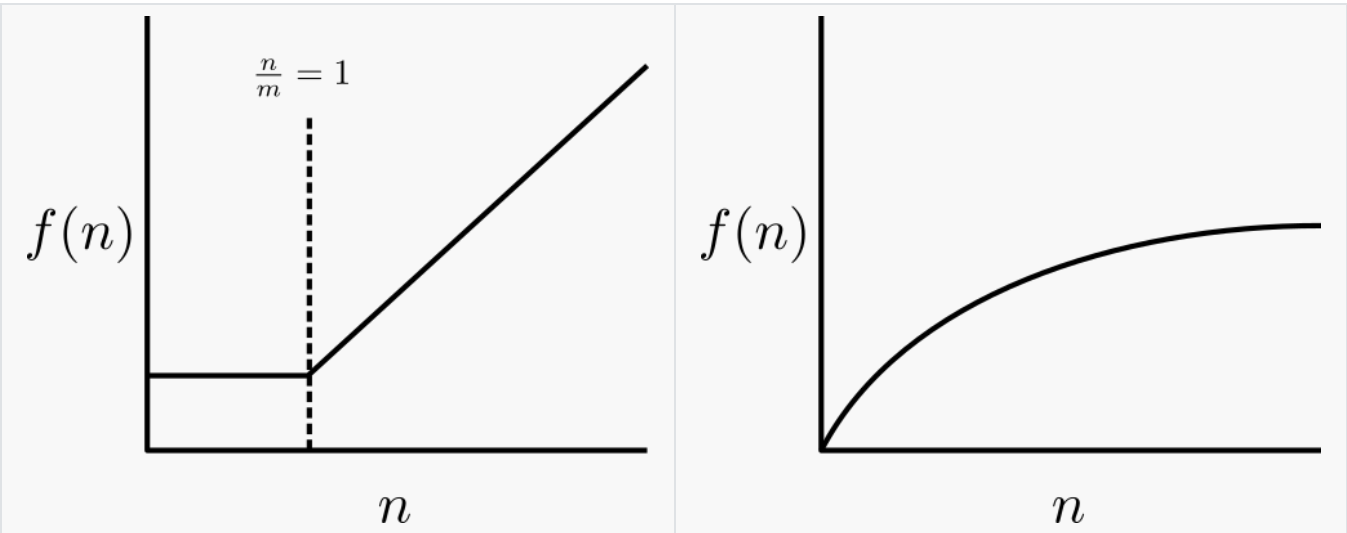
Recall that the hash table is not our only choice for a map implementation. While the associative array is a much poorer choice, the binary tree is a reasonable alternative to the hash map.



Which leaves an open question as to when should you choose a hash tree over a hash map.

The hash map has unbeatable performance when $\frac{n}{m} \approx 1$, but this assumes that we have some understanding and control within the problem space. m is generally chosen by us developers at design time before the software is even shipped to software users and n is generally decided by software users during run time rather than by us developers. If we do not understand the potential scale of n that users expect to use, it is difficult to select an appropriate m to take advantage of the efficiency of a hash table. It is also difficult to design a hash table capable of resizing itself to amortize the time complexity of insert like we did so with the doubling strategy when growing unordered arrays.

If we attempt to redefine m at run-time, we have to rehash everything in the hash table tanking our hash table efficiency to $O(n)$. For the array, we had everything to gain because insert was always $O(n)$ when the array was full so our performance was already at worst case. The hash table is generally operating closer to $O(1)$ until we reach a somewhat ambiguous threshold were the performance starts to trend toward $O(n)$; however, if we implement an aggressive strategy to redefine m then the hash table is guaranteed to operate at $O(n)$ with some frequency and is basically operating as an associative array. If there are better candidates out there that consistently perform better than $O(n)$ such as a binary tree map that operates at $O(\log_2 n)$, it raises the question of why design a complicated hash table that is no better than an associative array when there is a ready solution at hand that consistently performs more efficiently. A more appropriate way to frame this question is, when should we switch from a hash map to a binary tree map for a map implementation. It is critical to know your application's domain and its potential clients.



Hash Table Efficiency	BST Efficiency

If n remains a complete unknown, it is difficult to predict an appropriate m to take advantage of the hash map. If m is too small, the hash table will tend towards $O(n)$ efficiency for all operations because every operation depends on search and every linked list in the hash map contains some large proportion of n. If m is too large, the hash table is mostly empty and the space used is effectively wasted; however, it is still better to have a sparsely populated hash map servicing operations at $O(1)$ for high frequency searches than to choose too small an m. When n is an unknown, it is generally better to choose a binary tree map instead.

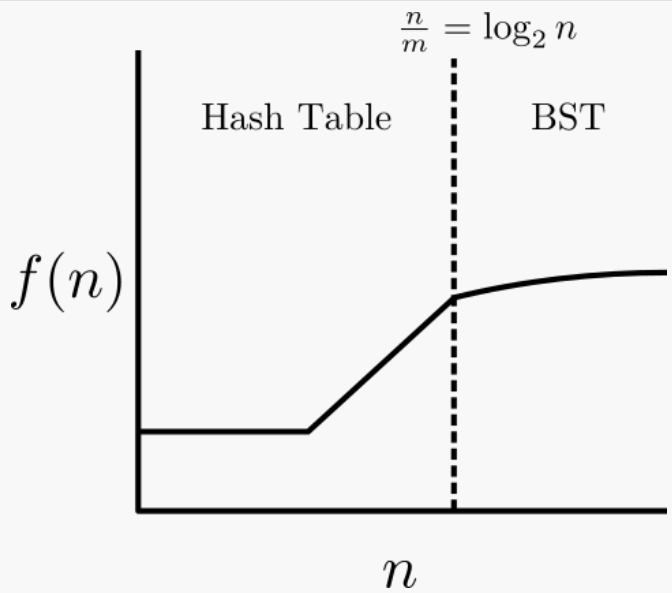
Consider the implication of the following relation in terms of a hash map's load factor with respect to a binary search tree of minimum height:

$$\frac{n}{m} = \log_2 n$$

The above equation marks the point at which a hash table performs the same as a binary search tree. If a hash table's load factor exceeds the maximum depth of a balanced binary search tree, then the binary search tree is a more efficient data structure in terms of both search and insert. The derivative of a linear equation is constant, so when a hash table begins to "fill up", the hash table becomes consistently less efficient. Counter-intuitively, the derivative of the logarithm decreases as n increases, so the binary search tree is extremely efficient when handling very large sets of data when compared to any linear data structures. This leads to the argument that if n is difficult to predict or unknowable, then it is better to chose a binary tree map over a hash map. However, if an m can be chosen with respect to n to maintain the relationship

$$\frac{n}{m} < \log_2 n$$

then a hash table would be the optimal solution.



Once $\frac{n}{m} > \log_2 n$, then the BST is a more efficient implementation of the map.

It might be difficult to anticipate $\frac{n}{m} = \log_2 n$, so a general guideline for selecting a hash table over a binary tree is to consider whether or not the size of the data n is bounded and can be anticipated. In other words, if n is predictable and of reasonable size, then it is possible to choose an m such that the hash table can provide constant time access; however, if n is generally an unknown and expected to be large, then a binary search tree is a better alternative by default.

Review

1. Describe the structure of a hash table.
2. What does the hashing function do in a hash table? How does the hashing function relate an arbitrary sized data representation to a fixed sized data representation? Practically, how does the hash function relate a key to a bucket?
3. What does bias mean in terms of the hashing function? If a hashing function is biased, what will the hash table generally look like? How will the efficiency of the hash table be affected?
4. Why is the number of buckets an independent variable with respect to the number of elements? Who controls the number of buckets and when is it chosen? Who controls the number of elements and when is the size of the input known?
5. What is the relationship between the number of buckets and the number of elements?
6. Why do we express the relationship between the number of buckets m and the number of elements n as $\frac{n}{m}$ in our analysis? In other words, why do we use a ratio, why is m in the denominator, and why is n in the numerator? What would swapping numerator and denominator make less sense?
7. Why do we condition our arguments with multiple cases in the hash table? In other words, why do we *caveat* our analysis with ideas like "where n is significantly larger than m" or "where n and m are approximately equal"?
8. Assuming an unbiased hash function, when n is significantly larger than m, what data structure does our hash table resemble, what is its efficiency for all map operations in this hash table, and why is it performing this way?
9. Assuming an unbiased hash function, when n is less than or approximately equal to m, what is its efficiency for all map operations and why is it performing this way?
10. What is the degenerate case for a hash table? Why might a degenerate case arise in a hash table? What is the efficiency of the degenerate case? What data structure does the hash table degenerate into in the degenerate case and why does it resemble this data structure?
11. Describe the implementation of a hash table for a map, *i.e.* a hash-map.
12. What is the efficiency of each map interface method for the hash-map?
13. In optimal conditions, the hash table is the most efficient map implementation. What are these optimal conditions?
14. Under what circumstances would the binary tree be the better solution for a map implementation? Why?
15. Why is the associative array generally the worst choice among the three potential map implementations discussed? Contrast the associative array's average performance with respect to the worst case performance of both the tree-map and hash-map.